Predicting Flood Probability

</> Code

Recovering a hidden sum from noisy risk factors — a worked case study on Kaggle's Playground Series S4E5

AUTHOR
Your Name

PUBLISHED
October 7, 2026

Interactive application

Open the Streamlit app

1 Introduction

Floods are the costliest recurring natural hazard in Vietnam — Typhoon Yagi alone (September 2024) flooded much of the north. Real flood forecasting needs rainfall, river gauges, and terrain data; this project tackles a much more modest, well-posed version of the question: given an area's scores on 20 flood-risk factors, how likely is it to flood?

The data comes from Kaggle's Playground Series S4E5: 1.1 million synthetic training rows, scored by R^2. It is a good portfolio problem for a reason that isn't obvious at first: the target turns out to be a simple function hidden under noise, and the whole project is about *finding that function* and then squeezing the last fraction of a percent out of the noise — on a leaderboard where 2,794 teams were separated by thousandths of a point.

The competition closed in May 2024, so every submission here is a late one: Kaggle scores it, but doesn't rank it. Scores are placed against the frozen final leaderboard instead (Section 6.3).

2 Data

The training set has **1,117,957 rows** and **20 integer features**, each a 0–10-ish risk score where higher always means more risk (e.g. a high `DrainageSystems` score means *worse* drainage). There are no missing values. The target, `FloodProbability`, takes only **83 distinct values**, all multiples of 0.005 — a strong hint that it was computed rather than measured.

3 The key discovery: the target is (originally) a sum

Playground datasets are generated synthetically from a small "original" dataset. This one's original — 50,000 rows, mirrored as `flood.csv` in naiyakhalid/flood-prediction-dataset — gives the game away:

▶ Code

```
max |FloodProbability - 0.005 x sum| over all 50,000 original rows: 1.1e-16
```

In the original data, `FloodProbability = 0.005 × (sum of the 20 factors)`**, exactly.** The synthetic generator then perturbed the factor values, which buries the same line in noise (Figure 1).

▶ Code

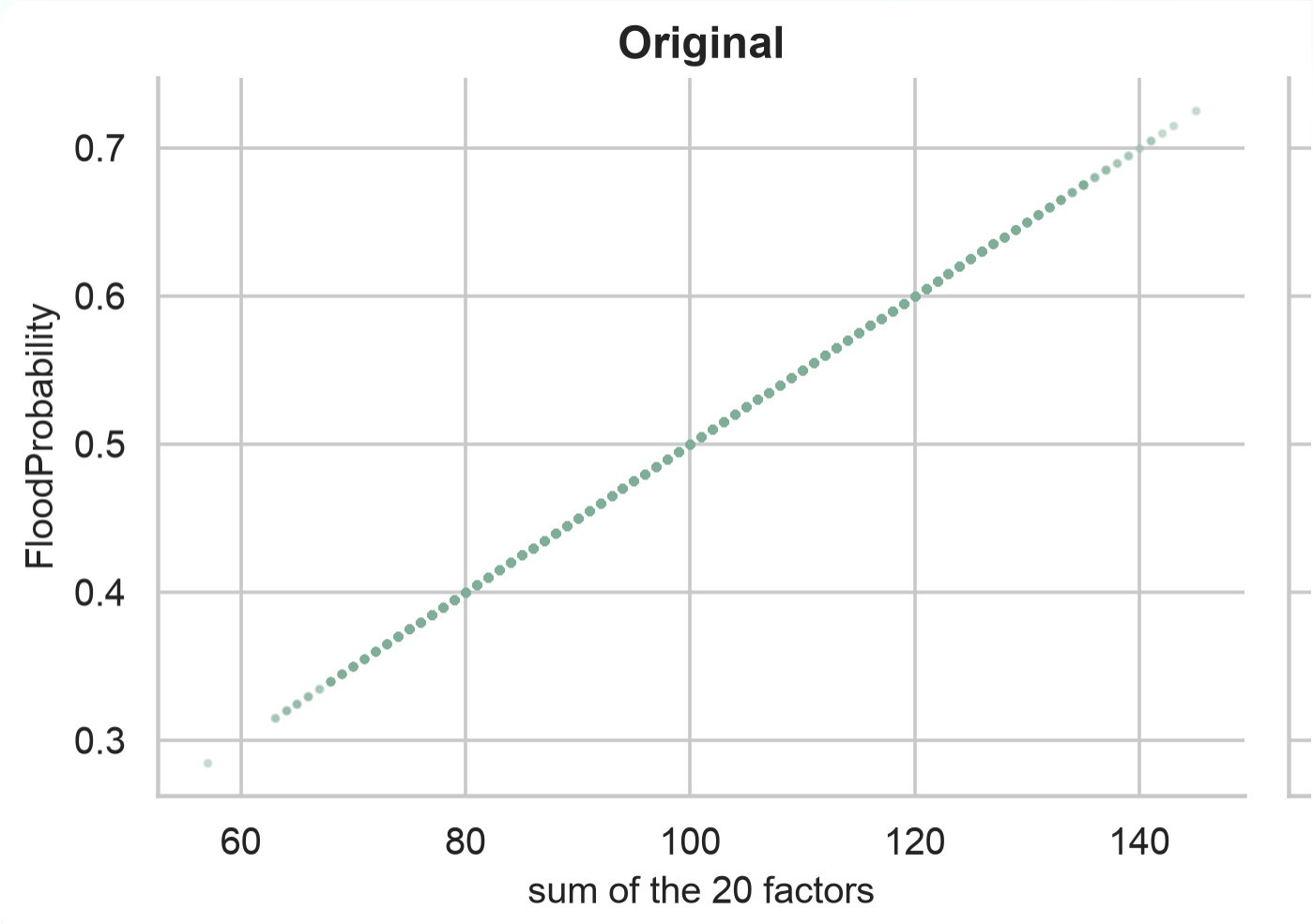


Figure 1: Target against the row sum. Left: the original data is a perfect line. Right: the competition's synthetic data (50,000-row sample) is the same relationship, blurred.

That reframes the competition: **given noisy factor values, recover the original row sum.** It also explains everything the data does next — why every factor correlates about equally and weakly with the target, why their sum correlates strongly, and why plain linear regression is an unusually strong baseline here (R^2 of about 0.845).

4 Feature engineering

If the target is a property of the whole row, the features should describe the whole row. `RowStatsFeatures` (in `src/flood_prediction/features.py`) keeps the 20 raw factors and adds:

- **11 row statistics:** sum, standard deviation, min, max, median, range, skewness, kurtosis, number of distinct values, and the 25th/75th percentiles.
- **The row's 20 values, sorted** (`sorted_00` = smallest … `sorted_19` = largest).

Everything added is invariant to *which* factor holds which value — exactly like the sum. The intuition: a row whose values are unusually spread out, or skewed, was probably perturbed more, so its observed sum is a less reliable estimate of the original one. The sorted values let a tree model learn those corrections directly.

5 Methodology

- **Validation.** Development decisions used a fixed 20% holdout (seed 42); every number that was submitted used K-fold out-of-fold (OOF) predictions on all 1.1M rows, where each row is predicted by the fold model that never saw it.
- **Models.** LightGBM, XGBoost, and CatBoost inside one scikit-learn pipeline with the feature step, so training and inference can't drift apart. Linear regression is the baseline.
- **Blending.** Non-negative least squares on the members' OOF predictions, normalised to sum to 1 — members can be dropped, never subtracted.
- **Two models, two jobs.** The app uses a single LightGBM (fast, explainable with SHAP); the leaderboard uses the blend.

6 Results

6.1 Feature sets and hyperparameters

These comparisons were run during development on the 20% holdout and recorded here (re-running them takes ~15 minutes, for no new information on every render):

experiment	holdout R^2
LightGBM, raw 20 factors only	0.84206
LightGBM, + row statistics	0.86839
LightGBM, + row statistics + sorted values	0.86860
… with colsample_bytree 0.5 (first guess)	0.86872
… with colsample_bytree 1.0, depth 10 (tuned)	0.86920
XGBoost, tuned (colsample 1.0, depth 10)	0.86918
CatBoost (depth 8)	0.86891
Blend of the three tuned models	0.86926

Two lessons stand out:

1. **Row statistics are worth +0.026 R^2** — by far the largest gain in the project, and a direct consequence of Section 3.
2. **Column sampling was the most important hyperparameter.** A common Kaggle default samples half the columns per tree; here that means half the trees can't see `row_sum`. Letting every tree see every column was worth +0.0005 — which, at this leaderboard's density, is the difference between rank ~500 and rank ~200.

6.2 What didn't help

Reported because a negative result is still a result:

- **Adding the 50,000 original rows to training** *hurt* (CatBoost holdout 0.86891 → 0.86838). They come from a different distribution — noise-free — so they teach the model the wrong noise level.
- **A neural network** (4-layer MLP on the same features, trained on the GPU) plateaued at R^2 0.864 and received zero blend weight: its errors were 98% correlated with the trees' and simply larger.
- **Extra row features** (value counts, higher moments, mode) moved holdout R^2 by less than 0.0001 — within noise.
- **Deeper CatBoost, slower learning rates**: within ±0.00003.

6.3 The leaderboard climb

submission	OOF R^2	public R^2	private R^2	would rank (private)	top %
v1	0.86916	0.86909	0.86870	#535–547	19.1%
v2	0.86940	0.86931	0.86893	#207–228	7.4%
v3	0.86943	0.86933	0.86893	#207–228	7.4%

The best submission scored **private R^2 0.86893**, which would have ranked **#207 of 2,794 teams (top 7.4%)**. For scale: rank 1 scored 0.86905, rank 50 scored 0.86899, and rank 500 scored 0.86872.

▶ Code

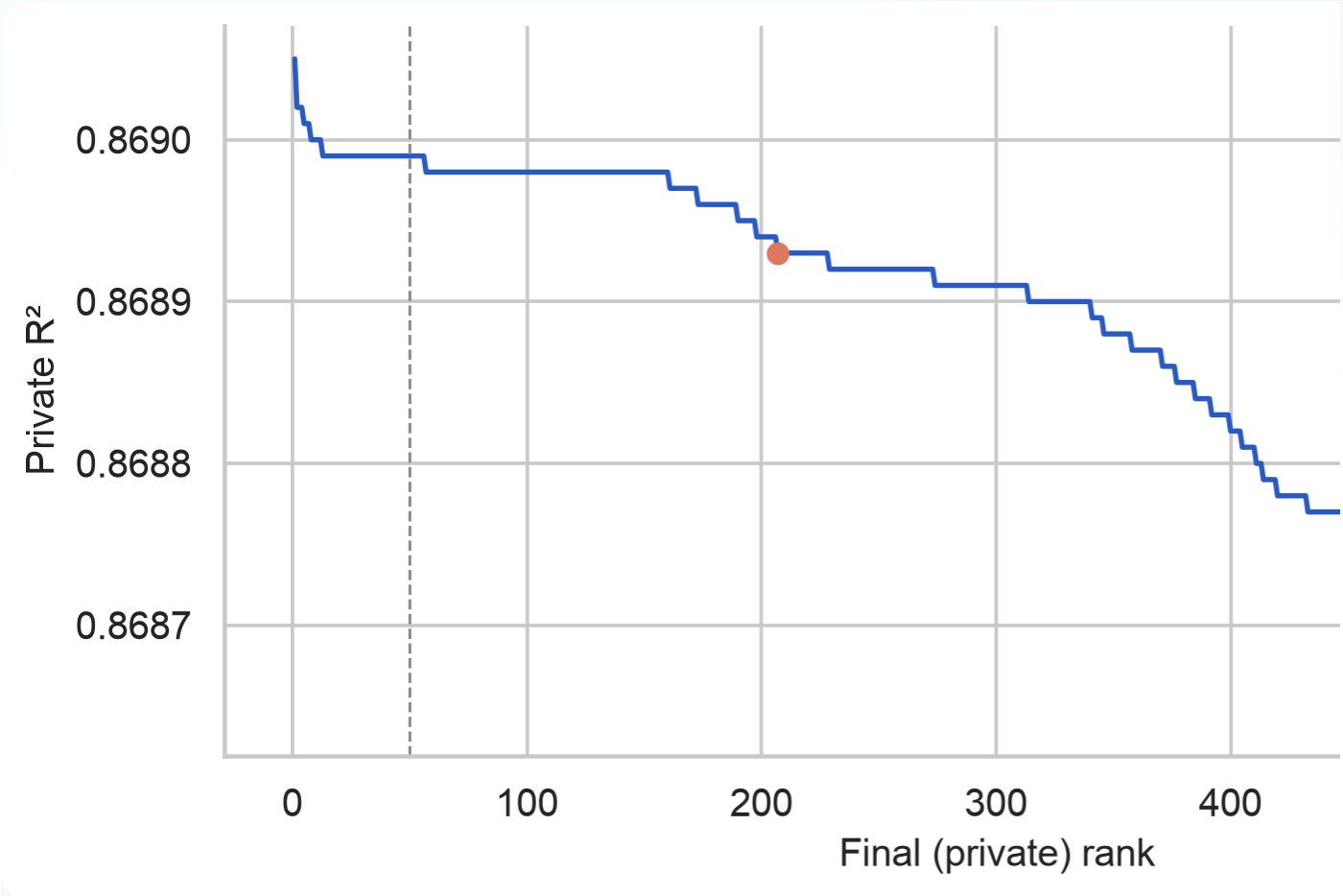


Figure 2: The top 600 of the final (private) leaderboard. Orange dots: this project's late submissions.

The board is extraordinarily dense: the entire top 500 fits inside a 0.0004-wide band of R^2. Joining the public and private boards by team shows what top-50 actually required — every one of the 56 teams that scored a private R^2 of 0.86899 or better had a public score of at least 0.86939. Most of those teams submitted the same widely shared public notebook, which averaged *other people's submission files* (including two AutoGluon runs). That approach was deliberately not used here:

everything in this project is trained from scratch, so the score reflects what the method in this repo actually does.

A consistent pattern worth knowing for anyone repeating this: this project's OOF R^2 overestimated the private score by a steady 0.00046–0.00050 on every submission — the private split is simply harder — so OOF was reliable for *ranking* changes even though it isn't on the same scale.

6.4 What the model learned (SHAP)

▶ Code

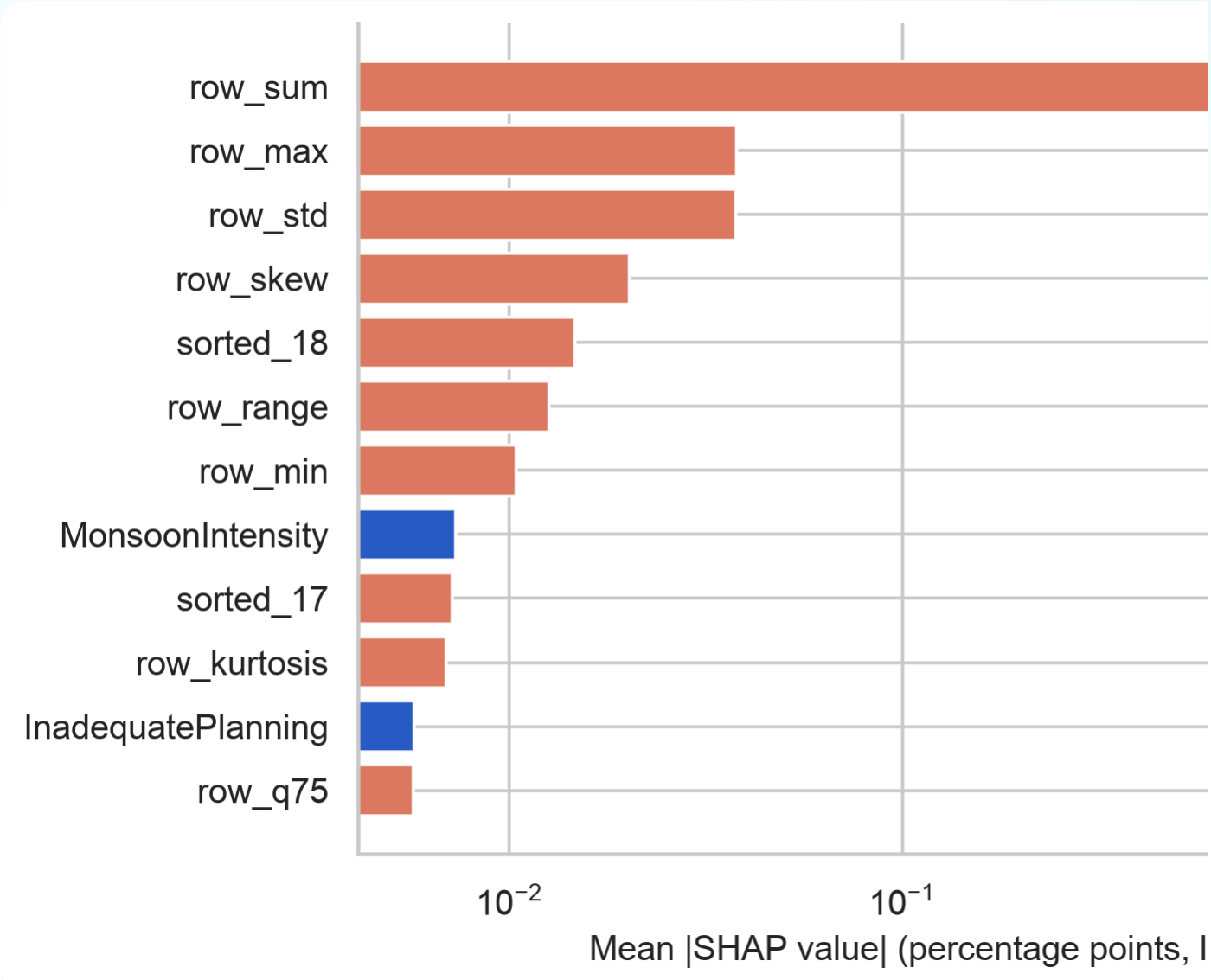


Figure 3: Top features of the app model (LightGBM) by mean |SHAP value| (log scale). Orange: engineered row statistics.

`row_sum` alone carries **93% of all the attribution** (note the log scale); what remains goes mostly to the spread statistics (max, std, skew) and the top sorted values, which correct the sum for how

much a row was perturbed. The named factors barely register on their own. That is the correct answer for this data, not a flaw: in a pure sum, every factor matters equally, so no single one stands out.

7 Limitations

- **Synthetic data.** The “flood probability” is a formula applied to synthetic scores, not observed floods. Nothing here says anything about real flood risk in Vietnam or anywhere else — the calculator in the app is a demonstration of the model, not a risk tool.
- **Late-submission ranks are hypothetical.** They compare a score against a frozen leaderboard; Kaggle breaks ties by earliest submission, which a late entry would always lose, so ranks are reported as the best position within a tie range.
- **Prior work was consulted.** After the first submission, the competition’s public notebooks were read for hyperparameter ideas (notably the column-sampling setting); features and code here are this project’s own.

8 Conclusion

The most valuable step in this project wasn’t modeling — it was checking the original dataset and discovering that the target is a sum. That one fact explained the feature engineering, the hyperparameter that mattered, the near-useless individual factors, and the SHAP picture. The rest was careful, honest measurement on a leaderboard where the differences that matter live in the fourth and fifth decimal place.